

1ª LISTA DE EXERCÍCIO

O objetivo deste exercício é a implementação de um programa que atenda aos requisitos abaixo e a criação de casos de testes para testar o programa. O resultado desta atividade deve ser um arquivo-texto com o programa e um documento com os casos de testes e explicações sobre eventuais decisões de implementação, como solicitado no texto abaixo.

O seu programa Python deve ser capaz de ler um arquivo CSV (*comma-separated values*), onde as linhas consistem de 3 valores: um nome, um CPF e um número de cartão de crédito. Para cada linha, verifique se todos os campos são não-vazios, e se o CPF e o número de cartão de crédito são válidos. Um CPF é considerado válido se ele tiver 11 dígitos com separadores (“.”, “-”, “/”) opcionais e se os dígitos satisfizerem o algoritmo de verificação de números de CPF (procure na Internet pela definição do algoritmo). Um número de cartão de crédito será válido se contiver 16 dígitos com possíveis espaços para separação e se os dígitos satisfizerem o algoritmo de Luhn (novamente, pesquise na Internet pela definição do algoritmo).

O programa deve imprimir na tela um relatório com os erros que forem achados. Cada linha do relatório deve ter a seguinte forma Linha NNN: Razão do erro, onde NNN é o número da linha com o erro formatado com espaço para quatro dígitos, e Razão do erro é uma string que começa com CPF — , se o erro foi no CPF, e CARTÃO — , se o erro foi no número do cartão de crédito. O resto da string deve descrever o erro. Para esta lista, considere que os erros são três: valor vazio para o campo, número incorreto de dígitos para o campo, e falha na verificação do(s) dígito(s) verificador(es). No primeiro caso, a string deve ser "Campo vazio", no segundo caso, "Quantidade incorreta de dígitos (NN dígitos)", onde NN é o número de dígitos do campo lido, e no último caso, "Erro nos dígitos verificadores".

Observe que uma dada linha pode conter erro no CPF **E** no número do cartão. Isso deve gerar **duas** linhas no relatório, uma para cada erro.

Não use nenhuma biblioteca externa, nem mesmo a *standard library* do Python. Procure usar o máximo possível de list comprehensions, sequências (em particular, as funções que geram sequências enumerate, zip, entre outras), geradores, e o que vimos em sala de aula. No documento, explique por que sua utilização é preferível à implementação equivalente convencional usada, por exemplo, em uma linguagem C.

Crie pelo menos 12 casos de teste com dados de CPF e número de cartão inválidos para verificar a correta execução do seu programa. Seja criativo com as razões dos dados não serem válidos. Imagine o que uma pessoa mais leiga da sociedade, talvez por ser mais idosa e sem muita experiência no uso de computadores e celulares, possa inserir (por exemplo, entrar o nome no lugar do CPF). Tente pensar o que uma pessoa mal-intencionada, por exemplo, um hacker, pode tentar usar como um dado para “quebrar” o seu programa intencionalmente. Explique por que cada caso de teste foi escolhido e que possível erro de implementação ele procura detectar. Para cada teste, registre entrada, resultado esperado e resultado obtido.